

An exploration of RNN behavior model of circuits

Patrick Huang

January 2026

1 Premise

This project is an exploration of modeling electronic circuits with recurrent neural networks.

Modeling a circuit as a *system*, we designate input and output quantities, using node voltages or loop currents. In classical circuit analysis, we use component models, circuit topology assumptions, and algebra to relate these values.

In this project, I use a RNN as the basis for the system model. Like before, the model relates inputs and outputs. For simplicity, I only use one input and output value per model. I use a circuit simulator to generate the “ground truth” data, which is used to train the NN. Then, I test the model on different input signals.

Because numerical circuit simulators do not always use constant time steps, the RNN is part of a differential equation: The neural network computes the *derivative* of some quantity, and we use numerical integration to compute the output, with the desired (possibly non-constant) time step.

2 Methods

2.1 Theoretical model

A traditional RNN is a set of two functions:

$$\begin{aligned} f : (\mathbf{R}^x, \mathbf{R}^h) &\rightarrow \mathbf{R}^h \\ g : \mathbf{R}^h &\rightarrow \mathbf{R}^y \end{aligned} \tag{1}$$

The RNN is run in a discrete system, with time index $n \in \mathbf{N}$. The input is a sequence of vectors $\mathbf{x}_n \in \mathbf{R}^x$, and the output $\mathbf{y}_n \in \mathbf{R}^y$. For each time step, the network generates an additional *hidden state* $\mathbf{h}_n \in \mathbf{R}^h$.

The function f computes the hidden state given the current input and the previous hidden state: $f(\mathbf{x}_n, \mathbf{h}_{n-1}) = \mathbf{h}_n$. The function g computes the output given the current hidden state: $g(\mathbf{h}_n) = \mathbf{y}_n$.

Because of the requirements of non uniform time steps described above, we modify the sys-

tem to be a *neural ordinary differential equation*. The function f , instead of computing the value of the next hidden state, outputs the *derivative* of the hidden state: $f(\mathbf{x}, \mathbf{h}) = \mathbf{h}'$. Then, the hidden state is numerically integrated, using the appropriate time step. As a consequence, the state vectors become continuous time.

The literature makes two additional changes: A feedback term τ is added to the function f , and the value $\mathbf{h}(0)$ is computed to satisfy $\mathbf{y}(0)$: $\mathbf{h}(0) = g^{-1}(\mathbf{y}(0))$. In this project, we simply make τ and $\mathbf{h}(0)$ learnable. Thus, the final system dynamics are:

$$\begin{aligned} \mathbf{h}(0) &= \dots \\ \mathbf{h}'(t) &= -\frac{1}{\tau}\mathbf{h}(t) + f(\mathbf{x}(t), \mathbf{h}(t)) \\ \mathbf{y}(t) &= g(\mathbf{h}(t)) \end{aligned} \tag{2}$$

$$\mathbf{h}(t + \Delta t) \approx \mathbf{h}(t) + \Delta t * \mathbf{h}'(t)$$

The algorithm that implements this system, computing the prediction time series given the input, is Algorithm 1.

2.2 Data generation

Each trained model is *circuit specific* — the learned behavior models a single circuit. That is, all parameters of the model (e.g. component values), except for the input and output, are frozen and learned. Training the model requires the input signal $\mathbf{x}(t)$, and the ground truth output $\mathbf{y}(t)$.

Using the circuit simulator LTSpice, we create a circuit of interest, and numerically simulate the transient behavior. Marking one node voltage as the input and another as the output, we obtain the two signals.

2.3 Training

To properly train a RNN, the forward pass must *run through multiple time steps* before performing a single update step, in order to learn the hidden state as a variable “system state” instead of an arbitrary input parameter. This is called “backpropagation through time”.

In the code, this is implemented by using a length of data in each forward pass: If the data sample is N time steps long, we simulate the model over those time steps, producing N output points. We compute the loss between the prediction and ground truth. Because (most of) the output points were *computed using results from past time steps*, the gradients propagate through the parameters that affect this time dependence — namely, the function f that updates the hidden state.

To improve the speed of training, the samples at the beginning of training are short, and are gradually increased to the final length (half the total ground truth length). This allows the model to quickly learn from a random initialization, before learning the long term dynamics. Of course, the end goal is to model both the short and long term accurately — shortening the examples at the beginning simply saves time. Figure 1 shows data samples of varying lengths throughout training.

The f function is a two layer MLP, and the g function a single layer. The model has around 1K parameters. We use MSE loss, and the Adam optimizer — standard approaches.

3 Results

3.1 RC filter

The RC filter circuit (Fig. 3) is a resistor and capacitor in series, with a time constant of 1ms. The input is the voltage on the left of the resistor, and the output is the voltage on the right of the resistor. As the results (Fig. 5) suggest, this circuit is a low pass filter.

The model was trained on a sine wave, and tested on sines of different frequency and amplitude, as well as a square wave. The model performs well, except for modeling the initial condition. As the square wave example shows, the circuit simulator had a different initial capacitor voltage. In the long term, the model matches the simulation.

3.2 Rectifier

The Rectifier circuit (Fig. 4) is a half bridge rectifier. The input is the voltage above the voltage source, and the output is the voltage above the capacitor. The model was trained on a sine wave, and tested on a higher frequency sine, and a square wave (Fig. 6).

The model learns the dynamics of the system quite well: Notice how in the high frequency sine wave, the model predicts that the peak voltage would be *less* than the low frequency sine, because there’s not as much time for the capacitor to charge in each cycle. In the square wave, the model deviates in the long term.

4 Appendix

- This project was inspired by the paper “Neural Ordinary Differential Equation Models of Circuits: Capabilities and Pitfalls”.
- The code and results are available at <https://github.com/phuang1024/RNNCircuit>

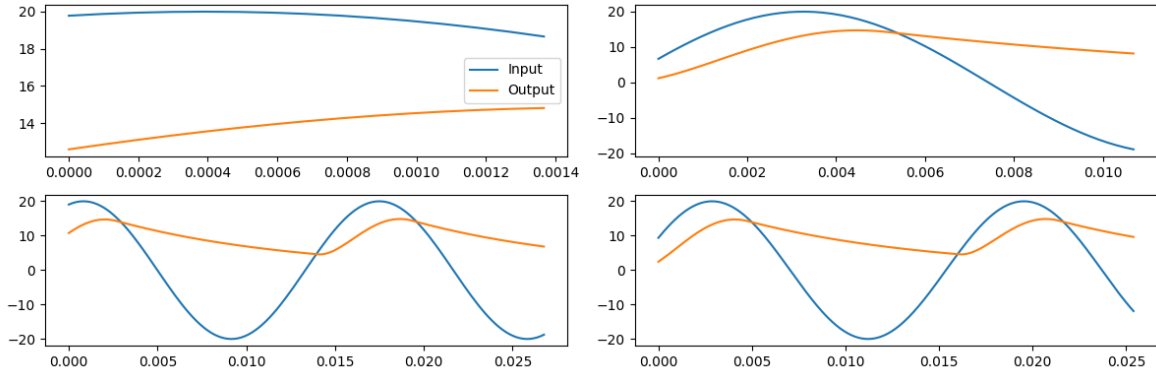


Figure 1: Samples of training data at various time scales.

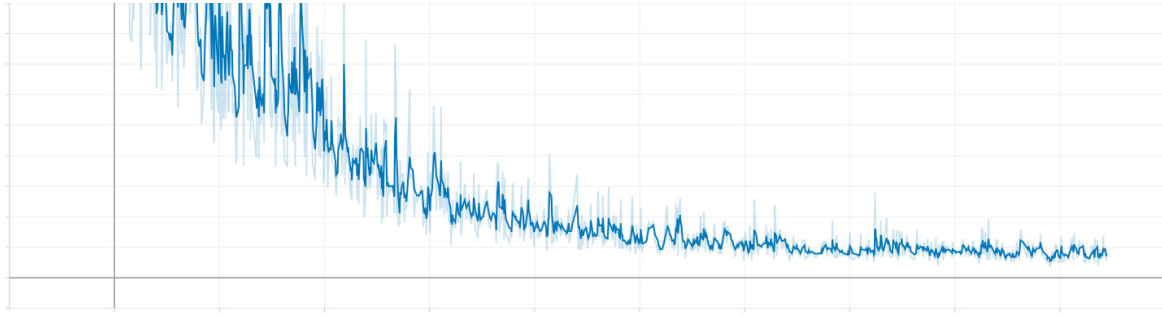


Figure 2: Training loss curve.

Algorithm 1 Model algorithm.

Require: h_0, τ learnable.

Require: f, g learnable.

Require: $x[n], dt[n]$ input time series.

Require: $\hat{y}[n]$ output time series.

$h \leftarrow h_0$

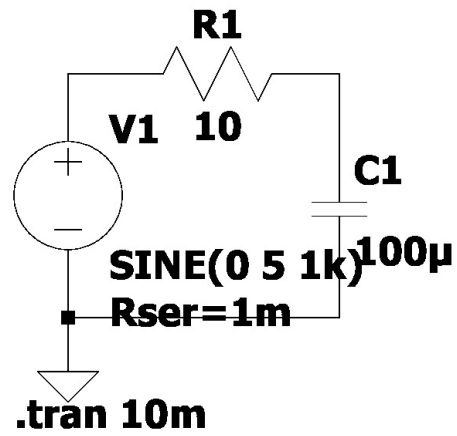
for $i = 0, 1, \dots, N - 1$ **do**

$dh \leftarrow -\frac{1}{\tau}h + f(x[i], h)$

$\hat{y}[i] \leftarrow g(h)$

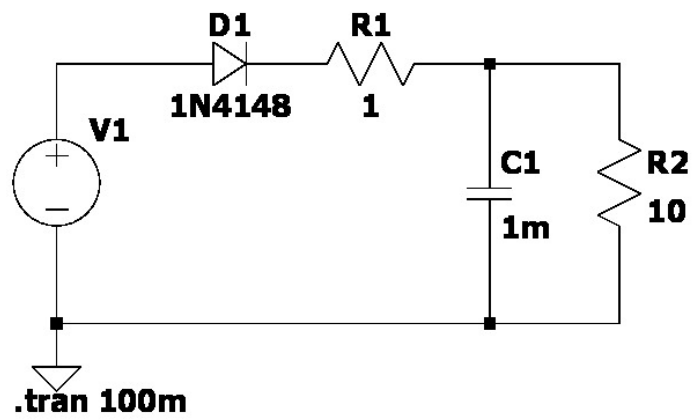
$h \leftarrow h + dh * dt[i]$

end for



--- C:\Users\huang\Work\Circuit\IRNN\RC.cas ---

Figure 3: RC circuit schematic.



--- C:\Users\huang\Work\Circuit\IRNN\Rectifier.cas ---

Figure 4: Rectifier circuit schematic.

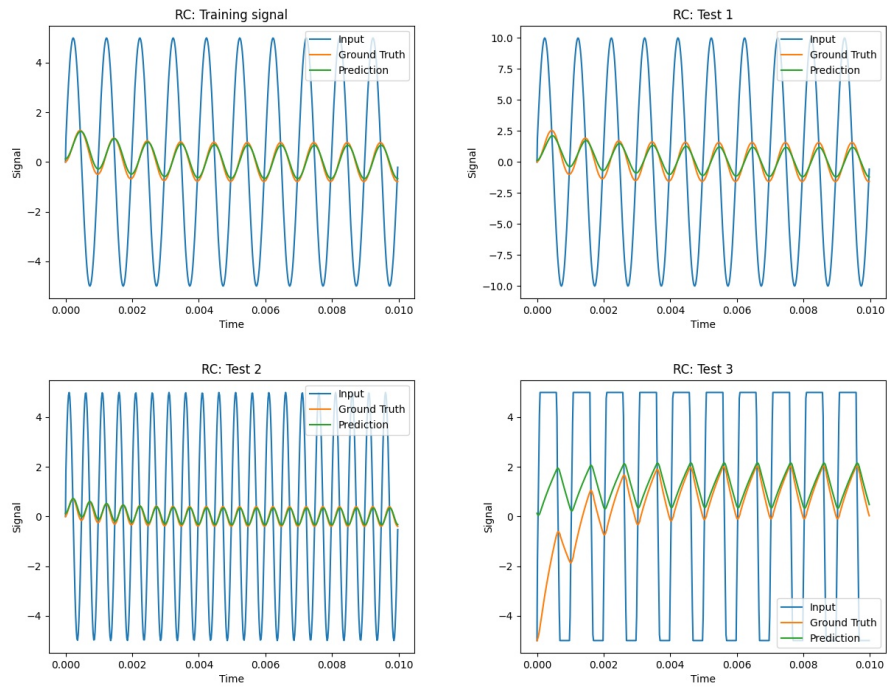


Figure 5: RC circuit results.

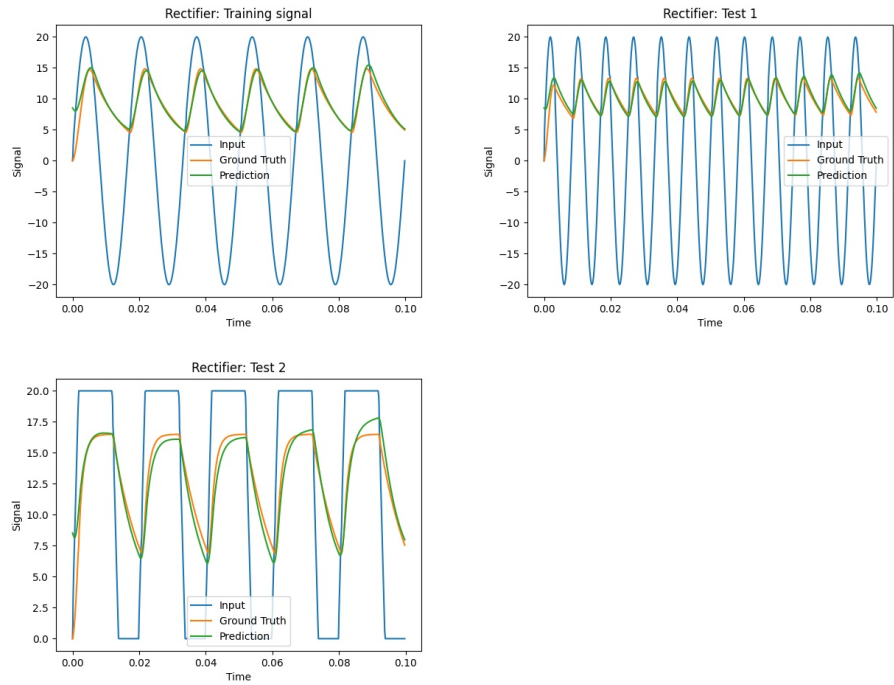


Figure 6: Rectifier circuit results.